

Building Subversive File Sharing With Client Side Applications

Building Subversive File Sharing With Client Side Applications - Author not stated, secttheory.com.

- Published: date not stated
- Original: <<http://www.secttheory.com/file-sharing.htm>>
- Preserved from: <http://www.secttheory.com/file-sharing.htm> (stored) on 2026-08-09
- Capture timestamp: 20080516202155
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

SecTheory - Internet Security

Building Subversive File Sharing With Client Side Applications

By Robert Hansen Date: 02/02/2008

Abstract: The subversive use of JavaScript has come in vogue over the last few years, since the invention of intranet hacking (Web Hacking 2.0). Soon after that was the introduction of the same concept but without the use of JavaScript (Web Hacking 1.0) to step backwards and reduce the usefulness of many client side protection systems. While the concept of intranet hacking using client side scripting is far from ubiquitous in the security world and needs continued research, there are many other implications for client side code. One concept that is possible is the subversive use of client side technologies to enable file sharing.

Overview: There are two basic premises of file sharing programs, the first being the ability to share files with one or more users elsewhere on the Internet. The second premise of file sharing is to reduce the bandwidth requirements of the person who has the file as they distribute it to the individuals who want the file in question. Both of these requirements can be met with the concept of subversive client side code and websites hosted by the colluding parties involved.

Scenario: There are three parties in this scenario who all take place in the file sharing. Two of whom are willing parties, and one of which is an unsuspecting party. All parties could represent many actual people/computers:

- Alice: Willing party (owns a web site, and has the original file).

- Bob & Zack: Willing parties (own web sites, and are interested in original file). Bob & Zack also represent more than one individual, but rather a set of people who are all interested in downloading the file in question.
- Cathy: Unwilling participant (owns only a browser and knows nothing about the original file).

Alice runs a website that Bob & Zack know contains the file they are interested in. Bob & Zack decide to download the known file from Alice and subsequently initiate a connection to a known program on Alice's website that advertizes this information. Information is then shared between the parties including cryptographic hashes of the files to ensure file integrity, the information on where Alice is supposed to send the file to and other potentially pertinent information.

Note: This information may also include a password to the encryption algorithm used if the information needs to remain secret from outside parties in transit. It is unlikely that public key cryptography would be used because for efficiency's sake all parties who receive the file must receive the same thing. The only caveat to this is if the set of all users who wanted the file in question never increased, allowing for a one time encryption with all the public keys of all the recipients in question before transfer.

Bob & Zack also run a similar program to Alice's on their own websites that will be used to eventually post the file to. This program does not have to reside anywhere in particular, but it must be known to Alice and eventually given to Cathy.

Note: Technically Bob & Zack don't need to run a web-server, just a passively listening socket that can eventually make outbound connections as well, but for ease of explanation let's assume Bob & Zack's programs are web servers with programs residing on their sites which respond to commands.

Alice somehow traps Cathy into visiting her website for a substantial duration of time. This could be because the website is interesting (IE: a flash game), it could be a framed page where the user never navigates away from the parent frame, or the user could simply be idle. Idle detection is relatively easy to detect based on a JavaScript timer, and it is possible that it would only send the data once the user had been idle for more than a certain pre-determined amount of time.

Note: It should be noted that cross site scripting could also be a good alternative, in that a user visiting an otherwise good page could be subverted into this process.

Once Cathy has visited Alice's page in question, several things happen. First, Alice may perform a load analysis on Cathy to test the viability of Cathy's browser as a transport mechanism (EG sufficient upload/download speeds). Once viability is verified, Alice delivers a payload to Cathy, which includes the first $N(1)$ bytes of the file in question that Bob & Zack want. Alice at the same time delivers a piece of JavaScript to Cathy which will serve as a client side command and control structure.

Once the software is running and the first N bytes are delivered, Cathy's browser is instructed to send it to Bob's website. Remember Bob & Zack represent many users who want the file in question; otherwise a simple GET request would be far better for bandwidth conservation for both Alice and the single other party.

Note: HTTP is not a particularly efficient protocol for sending files, due to the fact that it has lots of error checking. There may be other interesting ways to force other protocols cross domain, as Alice and Bob & Zack are colluding together against Cathy, but that is slightly outside the scope of this paper.

The JavaScript on Cathy's browser then creates a number of iframes corresponding to the set of users who want the file in question. Cathy's browser then, in JavaScript space writes into the iframes which then post themselves via cross site request forgery to both Bob & Zack's machines. Upon completion of receipt of the first $N(1)$ bytes Bob & Zack's machines then connect back to Alice's web site and confirm the receipt of the entire payload and ask for the next $N(2)$ bytes. Alice then sends Cathy an additional payload of the following $N(2)$ bytes, and the process continues.

Alternately, if something was damaged or broken during transmission, Bob & Zack can simply re-request the original $N(1)$ bytes. This can either be accomplished by asking Cathy's machine to re-send the original $N(1)$ bytes if her machine is still under subversive control (the information could simply be stored in memory for the duration of Cathy's persistence on the page in question). Alternately an additional compromise of another user's account can be used to send the following bytes. It's easy to see how the more users are compromised via the subversive JavaScript the faster the transmission of the file can become.

Accounting for error conditions is the largest hurdle to overcome in this system, but it could easily be overcome by a system of tracking and heavily error correction/checking upon sending and receipt, to insure that both Cathy and the ultimate recipients receive the file they were intended to. Bob & Zack both must not rely upon Cathy for anything, but must double check every chunk of bytes they receive with Alice to insure that the data has not been tampered with by Cathy, in the case where Cathy becomes aware of the compromise of her system.

Conclusion: The major advantage of this form of file sharing is that it utilizes third party bandwidth heavily, and doesn't require a complete compromise of their system. With very little effort a browser can be turned into a robot for transmission. In a way these users, when used heavily in parallel can be used as a virtual HTTP based network file system.